

PUNTO DE EJERCITACIÓN

Ejercitación

1. Cada participante, incluido el *host*, deberá crear un **módulo** cuyo nombre será su nombre y apellido en minúsculas, separados por un guion bajo.

Ejemplo: `mariana_perez.js`

2. Dentro de este módulo, cada integrante deberá definir un **array** que contenga la siguiente información personal:
 - a. Nombre completo
 - b. Pasatiempo
 - c. Lugar de residencia

Ejemplo:

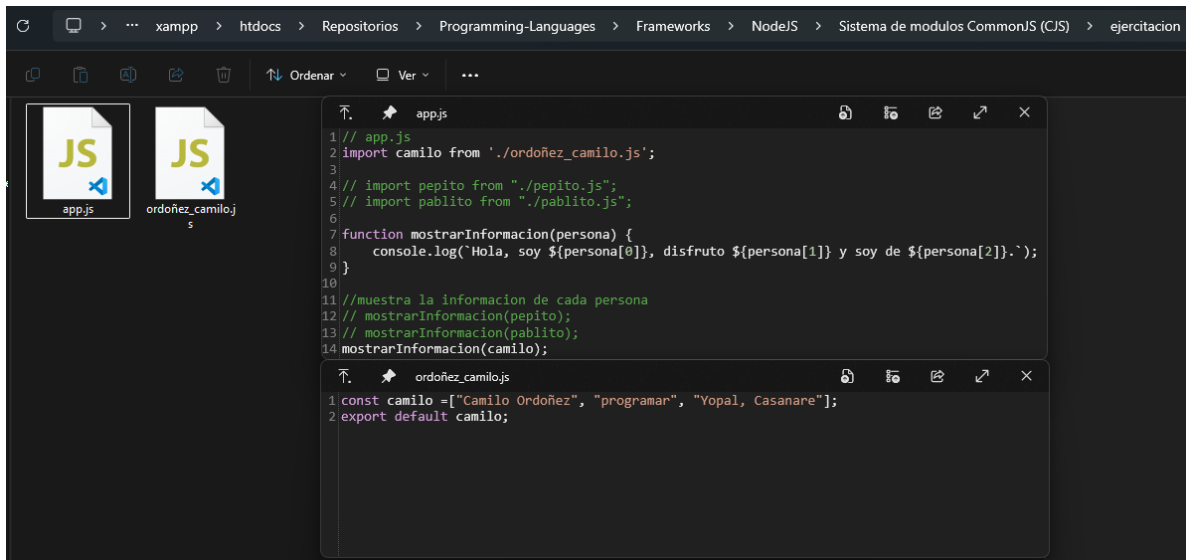
```
js
Copiar código

const mariana = ["Mariana Pérez", "ver series en Netflix", "Caballito, CABA"];
export default mariana;
```

3. Luego, en conjunto, deberán crear un archivo llamado `app.js`, en el que importarán todos los módulos creados anteriormente.
4. Posteriormente, también en conjunto, deberán programar una manera de mostrar en consola la información de cada uno de los participantes con el siguiente formato:

```
CSS
Copiar código

Hola, soy Mariana Pérez, disfruto viendo series en Netflix y soy de Caballito, CABA.
```



PRACTICA INTEGRADORA (MICRODESAFIOS I – II)

Práctica Integradora

Objetivo

Vamos a programar una calculadora sencilla con los conocimientos que hemos adquirido hasta el momento.

La calculadora contará con 4 funciones básicas:

- Sumar
- Restar
- Multiplicar
- Dividir



Micro desafíos - Paso I

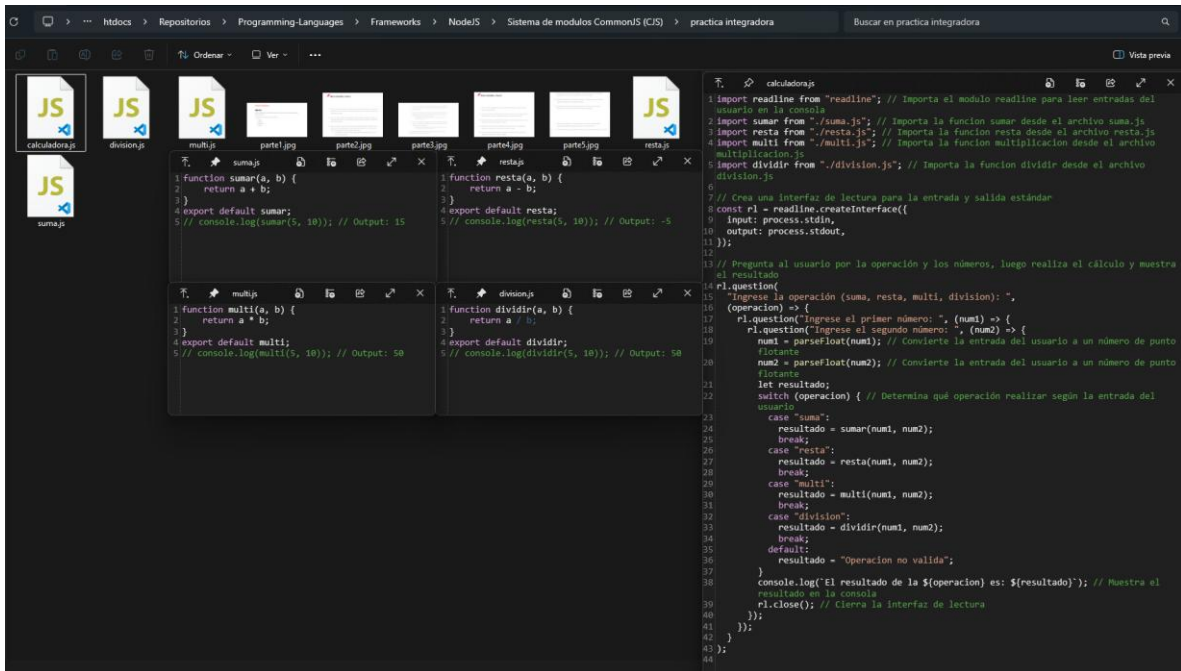
1. Crear un archivo `sumar.js` que contenga una función llamada *sumar*, la cual debe ser exportada al final del archivo. Esta función deberá recibir 2 parámetros y retornar la suma de los mismos.
2. Crear un archivo `restar.js` que contenga una función llamada *restar*, la cual debe ser exportada al final del archivo. Esta función deberá recibir 2 parámetros y retornar la resta de los mismos.
3. Crear un archivo `multiplicar.js` que contenga una función llamada *multiplicar*, la cual debe ser exportada al final del archivo. Esta función deberá recibir 2 parámetros y retornar la multiplicación de los mismos. Importante:
 - a. Contemplar el escenario donde si alguno de los dos parámetros es cero, la función retornará cero.
4. Crear un archivo `dividir.js` que contenga una función llamada *dividir*, la cual debe ser exportada al final del archivo. Esta función deberá recibir 2 parámetros y retornar la división de los mismos. Importante:
 - a. Contemplar el escenario donde si alguno de los dos parámetros es cero, la función retornará "No se puede dividir por cero".

Para verificar que hasta aquí viene todo bien, recomendamos probar cada una de las funciones y testear su correcto funcionamiento.

 Micro desafíos - Paso II

1. Crear un archivo calculadora.js en el cual deberemos requerir los cuatros archivos hechos con anterioridad.
2. Ejecutar la función que permite sumar y la función que permite restar, pasando como argumentos dos números cualesquiera. Mostrar en consola los resultados.
3. Ejecutar la función que permite multiplicar, pasando como argumentos dos números cualesquiera. Mostrar en consola el resultado.
4. Ejecutar la función que permite multiplicar, pasando ahora como uno de los dos argumentos, el número cero. Mostrar en consola el resultado.
5. Ejecutar la función que permite dividir, pasando como argumentos dos números cualesquiera. Mostrar en consola el resultado.
6. Ejecutar la función que permite dividir, pasando ahora como uno de los dos argumentos, el número cero. Mostrar en consola el resultado.

- a. ¿Qué hubiese sucedido si, en vez de generar un archivo por cada operación matemática, hubiésemos programado todo en un mismo archivo?
- b. ¿Por qué el mejor camino es generar distintos archivos y luego requerirlos en uno solo?



The screenshot shows a code editor with a project structure on the left and a main file, `calculadora.js`, on the right. The project structure includes files for `calculadora.js`, `division.js`, `multi.js`, `resta.js`, and `suma.js`. The `calculadora.js` file contains the following code:

```
1 import readline from "readline"; // Importa el modulo readline para leer entradas del
2 // usuario en la consola
3 import sumar from "./suma.js"; // Importa la funcion sumar desde el archivo suma.js
4 import resta from "./resta.js"; // Importa la funcion resta desde el archivo resta.js
5 import multi from "./multi.js"; // Importa la funcion multiplicacion desde el archivo
6 // multiplicacion.js
7 import dividir from "./division.js"; // Importa la funcion dividir desde el archivo
8 // division.js
9
10 // Crea una interfaz de lectura para la entrada y salida estándar
11 const rl = readline.createInterface({
12   input: process.stdin,
13   output: process.stdout,
14 });
15
16 // Pregunta al usuario por la operación y los números, luego realiza el cálculo y muestra
17 // el resultado
18 rl.question(
19   "Ingrese la operación (suma, resta, multi, division): ",
20   (operacion) => {
21     rl.question("Ingrese el primer número: ", (num1) => {
22       rl.question("Ingrese el segundo número: ", (num2) => {
23         num1 = parseFloat(num1); // Convierte la entrada del usuario a un número de punto
24         num2 = parseFloat(num2); // Convierte la entrada del usuario a un número de punto
25         let resultado;
26         switch (operacion) { // Determina qué operación realizar según la entrada del
27           // usuario
28           case "suma":
29             resultado = sumar(num1, num2);
30             break;
31           case "resta":
32             resultado = resta(num1, num2);
33             break;
34           case "multi":
35             resultado = multi(num1, num2);
36             break;
37           case "division":
38             resultado = dividir(num1, num2);
39             break;
40           default:
41             resultado = "Operación no válida";
42         }
43         console.log("El resultado de la $(operacion) es: ${resultado}"); // Muestra el
44         // resultado en la consola
45         rl.close(); // Cierra la interfaz de lectura
46       });
47     });
48   });
49 }
```

- a) Si todos los procesos se hubieran realizado en el mismo archivo hubiera habido inconvenientes si se llegaban a usar las mismas variables o asignación de nombres a procesos, además que al necesita un arreglo X proceso se puede volver tedioso ya que

deberá ver en qué parte se encuentra el “fallo” y arreglarlo sin ocasionar conflictos entre otros procesos.

- b) Principalmente porque se puede hacer recursividad llamando constantemente los procesos en los archivos y puntos necesarios, además que facilita tener “limpio” el código principal que unifica todos los procesos y en las carpetas o archivos dedicados se pueden tener procesos de organización distinto a los formatos principales.